

# Portfolio Site - Project Goal

Your Portfolio Site project will be a single complete submission that demonstrates every topic you covered in training through one integrated Django application. You will deliver a full stack portfolio website with a professional frontend built using HTML, Tailwind CSS, JavaScript, and Django Jinja2 templates. The website must include core public pages such as Home, About, Projects, and Contact, where visitors can view your profile, skills, and project work through clean responsive layouts. The Projects section must support slug based URLs, meaning each project will have its own dedicated detail page using Django URL dispatcher and routing configurations.

The backend must implement a complete authentication and authorization system with signup, login, logout, and protected dashboard access. Only authenticated users should be able to access the dashboard, and only the admin should have permission to add, update, or delete portfolio projects. This ensures you demonstrate real world access control using Django authentication features and ORM based backend design.

Your database layer must use PostgreSQL integrated into Django settings, with all models managed through migrations and accessible through the Django Admin Panel. You will create models such as Profile, Project, and ContactMessage, where projects include fields like title, description, tech stack, GitHub link, live demo link, created date, and slug. You must also show complete CRUD operations both through Django ORM queries and through the admin interface, including filtering, ordering, and search support.

The project must include a working contact form integrated with Django Email Service. When a visitor submits a message, the system should store it in the database and also send an email notification to the admin, along with a confirmation email back to the user. This deliverable proves your understanding of real communication workflows in web applications.

You must also implement Django REST Framework as part of the same project by exposing API endpoints for portfolio projects and contact submissions. The project must include serializers using ModelSerializer, validation logic, and authenticated API access through token based authentication. YourFR deliverables must include endpoints for fetching all projects, fetching a single project by slug, and creating new projects securely.

For DevOps coverage, you will containerize the complete application using Docker with a Dockerfile for the Django app and a docker-compose setup that runs both Django and PostgreSQL together. The project must support environment variables for secure configuration. You will also implement CI using GitHub Actions, where every push triggers automated testing, dependency installation, and Docker build verification. This ensures the project reflects modern deployment ready engineering standards.

Your final single date submission must include the full GitHub repository containing the Django source code, PostgreSQL integration, frontend templates, authentication system, admin managed CRUD features, email service, REST APIs, Docker configuration files, and GitHub Actions workflow. You must also provide a complete README documentation file explaining local setup, database configuration, Docker usage, API endpoint references, and deployment instructions. This single portfolio project will serve as your complete proof of Python, Django, DRF, PostgreSQL, CI, and Docker mastery in one production style application.

---

You will build a full stack Portfolio Website using Django.

This project will cover backend, frontend, database, authentication, REST APIs, email, Docker, and CI.

## 1. Core Features You Must Deliver

### A. Public Portfolio Website

Your site must include these pages.

#### Home Page

- Short intro
- Skills overview
- Call to action button

#### About Page

- Education
- Experience
- Resume download

### Projects Page

- List of projects with title, slug based URL
- Detail page for each project

### Contact Page

- Contact form with email integration
- Social links

## B. Authentication System

You must implement full authentication.

### Requirements

- Signup page
- Login page
- Logout
- Password validation
- Protected dashboard access

### Authorization

- Only admin can add projects
- Only logged in users can access dashboard

## 2. Database and Models Requirements

You must use PostgreSQL with Django ORM.

### Models to Create

#### User Profile Model

- Name
- Bio
- Profile image
- Skills list

#### Project Model

- Title
- Slug

- Description
- Tech stack
- GitHub link
- Live demo link
- Created date

#### Contact Message Model

- Name
- Email
- Message
- Timestamp

#### Optional Blog Model

- Title
- Slug
- Content
- Published date

#### Database Deliverables

- Postgres integration in settings
- Migrations applied
- Admin panel CRUD support

### 3. Django Admin Panel Deliverables

#### Admin Customization

- Manage projects from admin
- Manage contact messages
- Search and filter support

#### Admin Requirements

- Project listing with title and date
- Slug auto generation

### 4. Frontend Requirements

You must use these tools.

#### Templates

- Jinja2 based Django templates
- Base template with navbar and footer

#### Tailwind CSS (optional)

- Responsive design
- Clean UI
- Mobile friendly pages

#### HTML and JS

- Form validation using JavaScript
- Interactive project cards

### 5. CRUD Operations Deliverables

#### Projects CRUD

##### Create

- Add project from dashboard

##### Read

- Public projects listing
- Project detail by slug

##### Update

- Edit project details

##### Delete

- Remove projects with confirmation

#### ORM Usage

- QuerySets
- Filtering
- Ordering by date

### 6. Django Email Service Deliverables

#### Contact Form Email

## Requirements

- User submits message
- Admin receives email notification
- Confirmation email to user

## SMTP Setup

- Gmail SMTP or SendGrid support

## 7. Django REST Framework Deliverables

You must expose APIs.

### API Endpoints

#### Projects API

- GET all projects
- GET single project by slug
- POST new project for authenticated users

#### Contact API

- POST contact messages

#### Serializer Requirements

- ModelSerializer usage
- Field validation

#### Authentication for API (optional)

- Token based authentication

## 8. Deployment Ready DevOps Deliverables

### Docker Containerization

#### Requirements

- Dockerfile for Django app
- docker-compose.yml for Postgres + Django
- Environment variables support

#### CI Using GitHub Actions

#### Pipeline Requirements

- Run tests automatically
- Check code formatting
- Build Docker image

#### Example Workflow Steps

- Checkout code
- Install dependencies
- Run Django tests
- Build container

### 9. Final Deliverables Checklist

You must submit these items.

#### Code Deliverables

- Full Django project repository
- PostgreSQL integration
- REST API support
- Email service working

#### Documentation Deliverables

- README with setup steps
- API endpoint documentation
- Docker usage guide

#### Deployment Deliverables

- Dockerfile
- docker-compose.yml
- GitHub Actions workflow file

#### Portfolio Features Deliverables

- Responsive frontend
- Authentication system
- Admin managed project CRUD
- Contact form with email

## 10. Suggested Project Structure

```
portfolio_site/  
  accounts_app/  
  projects_app/  
  contact_app/  
  api_app/  
  templates/  
  static/  
  docker_setup/  
  .github/workflows/  
  manage.py
```